

Playwright

Modern Web automation for E2E, component and API testing across Chromium, Firefox and WebKit.

What it is and when to use it

Playwright is a modern Web automation framework suited to E2E, regression, smoke and cross-browser flows, with isolated contexts, parallel execution and rich diagnostics.

Official site and documentation

Use the official Playwright source as the primary reference: <https://playwright.dev/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

First test

Create a .spec.ts file, navigate with `page.goto`, prefer `getByRole/getByLabel` and assert with `expect`. Run `npx playwright test` and inspect the HTML report.

Locators and synchronization

Prefer semantic locators and web-first assertions. Avoid fixed sleeps; use Playwright auto-waiting and explicit state expectations.

Debug and evidence

Use UI Mode, Inspector, Trace Viewer, screenshots and video to identify product, environment, data or automation failures.

CI/CD

Install dependencies and browsers in CI, run the suite, and publish reports/traces/screenshots as artifacts.

Best practices and troubleshooting

Keep tests independent, use resilient locators, isolate data, control authentication state and investigate flaky tests rather than hiding them.

When automation is worth it

Prioritize scenarios that are repetitive, important, relatively stable and executed frequently. Smoke, critical regression and business flows are strong candidates. Exploration and unstable behavior usually need human judgment first.

From manual case to automation

Start from behavior: precondition, action and expected result. Then translate the scenario into navigation, locators and assertions. Good automation starts with good testing reasoning; code makes that reasoning repeatable.

Locators, auto-wait and assertions

Prefer `getByRole`, `getByLabel`, `getByText` and stable test ids. Auto-waiting and web-first assertions reduce fixed sleeps and make tests clearer and less flaky.

Debugging and evidence

UI Mode, Inspector, screenshots, video and Trace Viewer help reconstruct failures. Separate product defects from bad data, environment instability and automation defects.

Cross-browser and parallel execution

Projects can run the same suite on Chromium, Firefox and WebKit and model devices or environments. Parallelism speeds feedback but requires independent tests and isolated data.

API and test-data preparation

Playwright can combine UI and API flows. Use `APIRequestContext` to prepare preconditions, create or clean test data and validate backend responses without forcing every setup step through the browser. Keep test data deterministic and avoid coupling tests to shared mutable records.

CI/CD and maintenance

Install dependencies and browsers in CI, execute the suite and publish reports and traces. Treat flaky tests as technical debt instead of rerunning until green.

Installation — prerequisites

Install a supported Node.js version and validate `node --version` and `npm --version`. Create or enter the project directory before initializing Playwright.

Create a Playwright project

Run `npm init playwright@latest` and follow the wizard for JavaScript/TypeScript, test folder and CI options. Existing projects can install `@playwright/test` as documented.

Install browsers

Run `npx playwright install` to download browser binaries compatible with the installed Playwright version. Linux/CI may require additional dependencies.

Validate installation

Run `npx playwright test`. Use `npx playwright test --ui` when useful and `npx playwright show-report` for the HTML report.



Troubleshooting

If npx is not recognized, validate Node/npm and PATH. If browsers are missing, run `npx playwright install`.
Avoid copying node_modules across operating systems.